# Unit I
# Chapter 1: Introduction to Python

**Python**

Python is programming language that combines the features of C and Java. Python was developed by Guido Van Rossum in the year 1991. The name Python was picked from the TV show, Monty Python's Flying Circus. The logo of Python shows two intertwined snakes.

**Features of Python**

1. **Simple:** Python is simple programming language. It uses English like structures that make it easy to read and understand.
2. **Easy Language:** Python is an easy language. It uses very few keywords. It is easy to read, write, learn and understand.
3. **Open-Source:** Python is open-source and the community is always contributing to it to improve it. It is free and its source code is freely available to the public.
4. **High level language:** The Python language is designed to make developers life easy. Reading a Python code is like reading an English sentence.
5. **Dynamically-Typed Language:** Python is not statically-typed like Java. You don't need to declare data type while defining a variable.
6. **Platform-Independent:** Python is platform-independent. If you write a program, it will run on different platforms like Windows, Mac and Linux.
7. **Portable:** A program gives same result on any computer, which is known as portable.
8. **Procedural and Object-Oriented:** Python is object-oriented but supports both functional and object-oriented programming. Everything in Python is an object.
9. **Interpreted Language:** Python is an interpreted language. It comes with the IDLE (Interactive Development Environment). This is an interpreter and follows the REPL structure (Read-Evaluate-Print-Loop).
10. **Popular and Large Community Support:** Python has one of the largest communities on StackOverflow and Meetup. If you need help, the community will answer your questions.
11. **Large Standard Library:** The standard library is large and has many packages and modules with common and important functionality.
12. **Extensible:** Python is extensible. We can use and integrate code from other languages like C++ in your Python code.
13. **Embeddable:** It is also embeddable. We can insert Python programs into a C or C++ program.
14. **Scripting Language:** A scripting language is a programming language that does not use a compiler for executing the source code. Rather, it uses interpreter to translate the code to

SBC KARKALA

machine language. Python is considered as scripting language as it is interpreted and it is used on the internet to support other software.

15. **Database connectivity:** Python provides interfaces to connect its programs to all major databases like Oracle, Sybase or MySql.
16. **Scalable:** Python programs are scalable since they can run on any platform and use the features of the new platform effectively.
17. **Batteries included:** The huge library of Python contains several small applications which are already developed and immediately available to programmers. Eg: numpy, pandas.
18. **GUI Support:** You can use Python to create GUI (Graphical User Interfaces). You can use tkinter, PyQt, wxPython or Pyside for this.

**Flavors of python**

Flavors of Python simply refers to the different Python compilers. These flavors are useful to integrate various programming languages into Python. Let us look at some of these flavors :

**1. CPython :** CPython is the Python compiler implemented in C programming language. In this, Python code is internally converted into the byte code using standard C functions. Additionally, it is possible to run and execute programs written in C/C++ using CPython compiler.

**2. Jython :** Earlier known as JPython. Jython is an implementation of the Python programming language designed to run on the Java platform. Jython is extremely useful because it provides the productivity features of a mature scripting language while running on a JVM.

**3. PyPy :** This is the implementation using Python language. PyPy often runs faster than CPython because PyPy is a just-in-time(JIT) compiler while CPython is an interpreter.

**4. IronPython :** IronPython is an open-source implementation of the Python programming language which is tightly integrated with the .NET Framework.

**5. RubyPython :** RubyPython is a bridge between the Ruby and Python interpreters. It embeds a Python interpreter in the Ruby application's process using FFI (Foreign Function Interface).

**6. Pythonxy :** Python(x,y) is a free scientific and engineering development software for numerical computations, data analysis and data visualization based on Python.

**7. StacklessPython :** Small tasks which should run individually are called tasklets. Tasklets run independently on CPU and can communicate with others via channels. A channel is a manager that takes care of scheduling the tasklets,controlling them and suspending them. A thread is a process which runs hundreds of such tasklets. We can create threads and tasklets in StacklessPython which is reimplementation of original Python language.

**8. AnacondaPython :** When Python is redeveloped for handling large-scale data processing, predictive analytics and scientific computing, it is called Anaconda Python. This implementation mainly focuses on large scale of data.

SBC KARKALA

## Writing and Executing Python Program

```
#program to add two numbers
a=10
b=20
c=a+b
print("Sum is =",c)
```

The hash symbol '#' represents the comment line. a,b and c are variables. Suppose the python program is saved as sum.py. Here sum is the name and .py is the extension.

The execution of the Python program involves 2 Steps: **Compilation and Interpreter**

The program is converted into byte code by the Compiler. Byte code is a fixed set of instructions that represent arithmetic, comparison, memory operations, etc. It can run on any operating system and hardware. The byte code instructions are created in the .pyc file. The .pyc file is not explicitly created as Python handles it internally but it can be viewed with the following command:

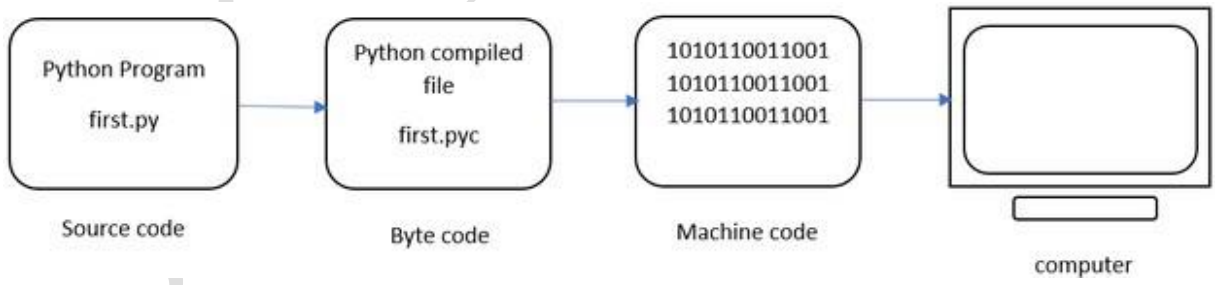
```
python -m py_compiler sum.py
```

The next step involves converting the byte code (.pyc file) into machine code. This step is necessary as the computer can understand only machine code (binary code). Python Virtual Machine (PVM) first understands the operating system and processor in the computer and then converts it into machine code. Further, these machine code instructions are executed by processor and the results are displayed.

However, the interpreter inside the PVM translates the program line by line thereby consuming a lot of time. To overcome this, a compiler known as Just In Time (JIT) is added to PVM. JIT compiler improves the execution speed of the Python program. This compiler is not used in all Python environments like CPython which is standard Python software.

To execute the sum.cpython-38.pyc we can use the following command:

```
python sum.cpython-38.pyc
```



# Chapter 2: Python Basics

## Identifiers

An **identifier** is a name given to a variable, function, class or module.

Rules for an Identifiers:

- Identifiers can be a combination of letters in lowercase (a to z) or uppercase (A to Z)or digits (0 to 9) or an underscore (_). Names like myCountry, other_1 and good_morning, all are valid examples.
- A Python identifier can begin with an alphabet or underscore(A–Z , a– z and _).
- An identifier cannot start with a digit. 1plus is invalid, but plus1 is valid.
- Keywords cannot be used as identifiers.
- One cannot use spaces and special symbols like !, @, #, $, % etc. as identifiers.
- Identifier can be of any length.

## Keywords

**Keywords** are a list of reserved words that have predefined meaning. It cannot be used as identifiers for variables, functions, constants or with any identifier name.

List of Keywords in Python

| | | |
|---|---|---|
| and | as | not |
| assert | finally | or |
| break | for | pass |
| class | from | nonlocal |
| continue | global | raise |
| def | if | return |
| del | import | try |
| elif | in | while |
| else | is | with |
| except | lambda | yield |
| False | True | None |

## Statements and Expressions

A **statement** is an instruction that the Python interpreter can execute.

For example, z = 1 is an assignment statement.

**Expression** is an arrangement of values and operators which are evaluated to make a new value. Expressions are statements as well.

For example,

```
>>> 8 + 2
10
```

SBC KARKALA

## Variables

Variable is a named placeholder to hold any type of data which the program can use to assign and modify during the course of execution. In Python, there is no need to declare a variable explicitly by specifying whether the variable is an integer or a float or any other type. To define a new variable in Python, we simply assign a value to a name.

## Legal Variable Names

Rules for creating legal variable names in Python:

• Variable names can consist of any number of letters, underscores and digits.

• Variable should not start with a number.

• Python Keywords are not allowed as variable names.

• Variable names are case-sensitive. For example, computer and Computer are different variables.

Naming conventions are not strictly enforced, it is considered a best practice to adhere to this convention.

- Python variables use lowercase letters with words separated by underscores as necessary to improve readability.
- Avoid naming a variable where the first character is an underscore.
- Ensure variable names are descriptive and clear enough. This allows other programmers to have an idea about what the variable is representing.

## Assigning Values to Variables

General form: **variable_name = expression**

The equal sign (=) also known as simple assignment operator is used to assign values to variables. In the general format, the operand to the left of the = operator is the name of the variable and the operand to the right of the = operator is the expression which can be a value or any code snippet that results in a value.

Example:

```
>>> number =100
>>> number
100

>>> miles =1000.0
>>> miles
1000.0
```

SBC KARKALA

```
>>> name ="Python"
>>> name
'Python'
```

Python allows you to assign a single value to several variables simultaneously. For example,

```
>>> a = b = c =1
```

**Operators**

Operators are symbols, such as +, –, =, >, and <, that perform certain mathematical or logical operation to manipulate data values and produce a result based on some rules. An operator manipulates the data values called operands.

Consider the expression,

```
>>> 4 + 6
```

where 4 and 6 are operands and + is the operator.

Python language supports a wide range of operators. They are

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Bitwise Operators

**1. Arithmetic Operators**

Arithmetic operators are used to execute arithmetic operations such as addition, subtraction, division, multiplication etc.

List of Arithmetic Operators *Note:* The value of p is 2 and q is 3.

| Operator | Operator Name | Description | Example |
|---|---|---|---|
| + | Addition operator | Adds two operands, producing their sum. | p + q = 5 |
| – | Subtraction operator | Subtracts the two operands, producing their difference. | p – q = –1 |
| * | Multiplication operator | Produces the product of the operands. | p * q = 6 |
| / | Division operator | Produces the quotient of its operands where the left operand is the dividend and the right operand is the divisor. | q / p = 1.5 |
| % | Modulus operator | Divides left hand operand by right hand operand and returns a remainder. | q % p = 1 |
| ** | Exponent operator | Performs exponential (power) calculation on operators. | p**q = 8 |
| // | Floor division operator | Returns the integral part of the quotient. | 9//2 = 4 and 9.0//2.0 = 4.0 |

BC KARKALA

Example:

```
>>> 10+35
45
>>> -10+35
25
>>> 4*2
8
>>> 4**2
16
>>> 45/10
4.5
>>> 45//10.0
4.0
>>> 2025%10
5
>>> 2025//10
202
```

## 2. Assignment Operators

Assignment operators are used for assigning the values generated after evaluating the right operand to the left operand. Assignment operation always works from right to left. Simple assignment is done with the equal sign (=).

Compound assignment operators support shorthand notation for avoiding the repetition of the left-side variable on the right side. Compound assignment operators combine assignment operator with another operator with = being placed at the end of the original operator.

For example, the statement
>>> x = x + 1
can be written in a compactly form as shown below.
>>> x += 1

List of Assignment Operators

| Operator | Operator Name | Description | Example |
|---|---|---|---|
| = | Assignment | Assigns values from right side operands to left side operand. | z = p + q assigns value of p + q to z |
| += | Addition Assignment | Adds the value of right operand to the left operand and assigns the result to left operand. | z += p is equivalent to z = z + p |
| -= | Subtraction Assignment | Subtracts the value of right operand from the left operand and assigns the result to left operand. | z -= p is equivalent to z = z - p |
| *= | Multiplication Assignment | Multiplies the value of right operand with the left operand and assigns the result to left operand. | z *= p is equivalent to z = z * p |
| /= | Division Assignment | Divides the value of right operand with the left operand and assigns the result to left operand. | z /= p is equivalent to z = z / p |
| **= | Exponentiation Assignment | Evaluates to the result of raising the first operand to the power of the second operand. | z**= p is equivalent to z = z ** p |
| //= | Floor Division Assignment | Produces the integral part of the quotient of its operands where the left operand is the dividend and the right operand is the divisor. | z //= p is equivalent to z = z // p |
| %= | Remainder Assignment | Computes the remainder after division and assigns the value to the left operand. | z %= p is equivalent to z = z % p |

VARKALA

Example:

```
>>> x = 5
>>> x = x + 1
>>> x
6
```

```
>>> p = 10
>>> q = 12
>>> q += p
>>> q
22
```

```
>>> q *= p
>>> q
220
```

### 3. Comparison Operators

When the values of two operands are to be compared then comparison operators are used. The output of these comparison operators is always a Boolean value, either True or False.

List of Comparison Operators *Note:* The value of p is 10 and q is 20.

| Operator | Operator Name | Description | Example |
|---|---|---|---|
| == | Equal to | If the values of two operands are equal, then the condition becomes True. | (p == q) is not True. |
| != | Not Equal to | If values of two operands are not equal, then the condition becomes True. | (p != q) is True |
| > | Greater than | If the value of left operand is greater than the value of right operand, then condition becomes True. | (p > q) is not True. |
| < | Lesser than | If the value of left operand is less than the value of right operand, then condition becomes True. | (p < q) is True. |
| >= | Greater than or equal to | If the value of left operand is greater than or equal to the value of right operand, then condition becomes True. | (p >= q) is not True. |
| <= | Lesser than or equal to | If the value of left operand is less than or equal to the value of right operand, then condition becomes True. | (p <= q) is True. |

Example:

```
>>>10 == 12
False
>>>10 != 12
True
>>>10 < 12
True
>>>10 > 12
False
>>>10 <= 12
True
```

```
>>>10 >= 12
False
>>> "P" < "Q"
True
>>> "Aston" > "Asher"
True
>>> True == True
True
```

## 4. Logical Operators

The logical operators are used for comparing or negating the logical values of their operands and to return the resulting logical value. The values of the operands on which the logical operators operate evaluate to either True or False. The result of the logical operator is always a Boolean value, True or False.

List of Logical Operators

| Operator | Operator Name | Description |
|---|---|---|
| and | Logical AND | Performs AND operation and the result is True when both operands are True |
| or | Logical OR | Performs OR operation and the result is True when any one of both operand is True |
| not | Logical NOT | Reverses the operand state |

Boolean Logic Truth Table

| P | Q | P and Q | P or Q | Not P |
|---|---|---|---|---|
| True | True | True | True | False |
| True | False | False | True | |
| False | True | False | True | True |
| False | False | False | False | |

Example:

```
>>> True and False
False
>>> True or False
True
>>> not(True) and False
False
>>> not(True and False)
True
```

```
>>> (10 < 0) and (10 > 2)
False
>>> (10 < 0) or (10 > 2)
True
>>> not(10 < 0) or (10 > 2)
True
>>> not(10 < 0 or 10 > 2)
False
```

## 5. Bitwise Operators

Bitwise operators treat their operands as a sequence of bits (zeroes and ones) and perform bit by bit operation.

SBC KARKALA

## List of Bitwise Operators

*Note:* The value of p is 60 and q is 13.

| Operator | Operator Name | Description | Example |
|---|---|---|---|
| & | Binary AND | Result is one in each bit position for which the corresponding bits of both operands are 1s. | p & q = 12 (means 0000 1100) |
| \| | Binary OR | Result is one in each bit position for which the corresponding bits of either or both operands are 1s. | p \| q = 61 (means 0011 1101) |
| ^ | Binary XOR | Result is one in each bit position for which the corresponding bits of either but not both operands are 1s. | (p ^ q) = 49 (means 0011 0001) |
| ~ | Binary Ones Complement | Inverts the bits of its operand. | (~p) = –61 (means 1100 0011 in 2's complement form due to a signed binary number. |
| << | Binary Left Shift | The left operands value is moved left by the number of bits specified by the right operand. | p << 2 = 240 (means 1111 0000) |
| >> | Binary Right Shift | The left operands value is moved right by the number of bits specified by the right operand. | p >> 2 = 15 (means 0000 1111) |

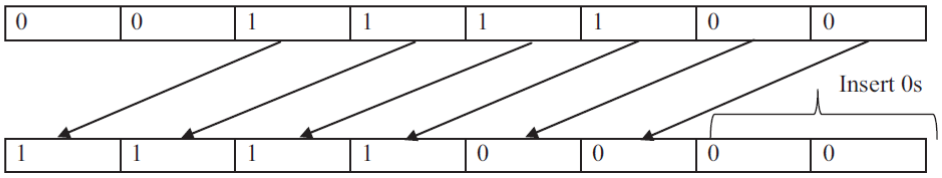
## Bitwise Truth Table

| P | Q | P & Q | P \| Q | P ^ Q | ~ P |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 | |
| 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | |

Example:

| Bitwise and ( & ) | | | | Bitwise or ( \| ) | | | |
|---|---|---|---|---|---|---|---|
| a | = | 0011 1100 | → (60) | a | = | 0011 1100 | → (60) |
| b | = | 0000 1101 | → (13) | b | = | 0000 1101 | → (13) |
| a & b | = | 0000 1100 | → (12) | a \| b | = | 0011 1101 | → (61) |
| **Bitwise exclusive or ( ^ )** | | | | **One's Complement ( ~ )** | | | |
| a | = | 0011 1100 | → (60) | a | = | 0011 1100 | → (60) |
| b | = | 0000 1101 | → (13) | ~ a | = | 1100 0011 | → (-61) |
| a ^ b | = | 0011 0001 | → (49) | | | | |
| **Binary left shift ( << )** | | | | **Binary right shift ( >> )** | | | |
| a | = | 0011 1100 | → (60) | a | = | 0011 1100 | → (60) |
| a << 2 | = | 1111 0000 | → (240) | a >> 2 | = | 0000 1111 | → (15) |
| left shift of 2 bits | | | | right shift of 2 bits | | | |

FIGURE 2.2 shows how the expression 60 << 2 would be evaluated in a byte.




Example:

```
>>> p =60
>>> p << 2
240
>>> p = 60
>>> p >> 2
15
```

```
>>> q = 13
>>> p & q
12
>>> p | q
61
```

```
>>> ~p
–61
>>> p << 2
240
>>> p >> 2
15
```

**Precedence and Associativity**

Operator precedence determines the way in which operators are parsed with respect to each other. Operators with higher precedence become the operands of operators with lower precedence. Associativity determines the way in which operators of the same precedence are parsed. Almost all the operators have left-to-right associativity.

```
>>> 2 + 3 * 6
20
>>> (2 + 3) * 6
30
```

First perform * and then + -> 2 + 18 - > 20

First perform + and then * -> 5 * 6 - > 30

Operator Precedence in Python

| Operators | Meaning |
|---|---|
| () | Parentheses |
| ** | Exponent |
| +x, –x, ~x | Unary plus, Unary minus, Bitwise NOT |
| *, /, //, % | Multiplication, Division, Floor division, Modulus |
| +, – | Addition, Subtraction |
| <<, >> | Bitwise shift operators |
| & | Bitwise AND |
| ^ | Bitwise XOR |
| \| | Bitwise OR |
| ==, !=, >, >=, <, <=, is, is not, in, not in | Comparisons, Identity, Membership operators |
| not | Logical NOT |
| and | Logical AND |
| or | Logical OR |

KARKALA

## Data Types

Data types specify the type of data like numbers and characters to be stored and manipulated within a program. Basic data types of Python are **Numbers, Boolean, Strings, None.**

### 1. Numbers

Integers, floating point numbers and complex numbers fall under Python numbers category. They are defined as int, float and complex class in Python.

Integers can be of any length; it is only limited by the memory available. A floating point number is accurate up to 15 decimal places. Integer and floating points are separated by decimal points. 1 is an integer, 1.0 is floating point number. Complex numbers are written in the form, x + yj, where x is the real part and y is the imaginary part.

### 2. Boolean

Boolean datatype represents one of the two values i.e. True or False. It is used to represent the truth values of the expressions. For example, 1==1 is True whereas 2<1 is False. The Boolean values, True and False are treated as reserved words. The output <class 'bool'> indicates the variable is a boolean data type.

### 3. Strings

A string consists of a sequence of one or more characters, which can include letters, numbers, and other types of characters. A string can also contain spaces. We can use single quotes or double quotes to represent strings and it is also called a string literal. Multiline strings can be denoted using triple quotes, ''' or """.

For example,

```
>>> s = 'This is single quote string'
>>> s = "This is double quote string"
>>> s = '''This is
        Multiline
                string'''
>>> s = "a"
```

### 4. None

None is another special data type in Python. None is frequently used to represent the absence of a value.

For example,

```
>>> money = None
```

None value is assigned to variable money.

SBC KARKALA

**Indentation**

In Python, Programs get structured through indentation. Usually, we expect indentation from any program code, but in Python it is a requirement and not a matter of style. This principle makes the code look cleaner and easier to understand and read. Any statements written under another statement with the same indentation is interpreted to belong to the same code block. If there is a next statement with less indentation to the left, then it just means the end of the previous code block. Usually, four whitespaces are used for indentation and are preferred over tabs. Incorrect indentation will result in Indentation Error.

**Comments**

Comments are an important part of any program. A comment is a text that describes what the program or a particular part of the program is trying to do and is ignored by the Python interpreter. Comments are used to help you and other programmers understand, maintain, and debug the program. **Python uses two types of comments**: single-line comment and multiline comments.

**i) Single Line Comment**

In Python, use the hash (#) symbol to start writing a comment. Hash (#) symbol makes all text following it on the same line into a comment. For example,

```
#This is single line Python comment
```

ii) **Multiline Comments**

If the comment extends multiple lines, then one way of commenting those lines is to use hash (#) symbol at the beginning of each line.

For example,

```
#This is
#multiline comments
#in Python
```

Another way of doing this is to use triple quotes, either ''' or """. These triple quotes are generally used for multiline strings. However, they can be used as a multiline comment as well.

For example,

```
'''This is
multiline comment
in Python using triple quotes'''
```

SBCKARKALA

**Reading Input**

In Python, input() function is used to gather data from the user. The syntax for input function is,

**variable_name = input([prompt])**

prompt is a string written inside the parenthesis that is printed on the screen. The prompt statement gives an indication to the user of the value that needs to be entered through the keyboard. When the user presses Enter key, the program resumes and input returns what the user typed as a string. Even when the user inputs a number, it is treated as a string which should be casted or converted to number explicitly using appropriate type casting function.

```
>>> person = input("What is your name?")
What is your name? Carrey
>>> person
'Carrey'
```

**Print Output**

The print() function allows a program to display text onto the console. The print function will print everything as strings and anything that is not already a string is automatically converted to its string representation. For example,

```
>>> print("Hello World!!")
Hello World!!
```

Two major string formats which are used inside the print() function:

1. str.format()
2. f-strings

**1. str.format() Method**

The format() method formats the specified value(s) and insert them inside the string's placeholder. The placeholder is defined using curly brackets: {}
The format() method returns a new string with inserted values.
The syntax for format() method is,

```
str.format(p0, p1, ..., k0=v0, k1=v1, ...)
```

where p0, p1,... are called as positional arguments and, k0, k1,... are keyword arguments with their assigned values of v0, v1,... respectively.

SBC KARKALA

Positional arguments are a list of arguments that can be accessed with an index of argument inside curly braces like {index}. Index value starts from zero.

Keyword arguments are a list of arguments of type keyword = value, that can be accessed with the name of the argument inside curly braces like {keyword}.

**Example: 1**

```
country = input("Which country do you live in?")
print("I live in {0}".format(country))
```

**Output:**

Which country do you live in? India

I live in India

**Example 2:**

**Program 2.2: Program to Demonstrate the Positional Change of Indexes of Arguments**

```
1. a = 10
2. b = 20
3. print("The values of a is {0} and b is {1}".format(a, b))
4. print("The values of b is {1} and a is {0}".format(a, b))
```

OUTPUT

```
The values of a is 10 and b is 20
The values of b is 20 and a is 10
```

**Example 3:**

```
1. >>> print("Give me {ball} ball".format(ball = "tennis"))
   Give me tennis ball
```

The keyword argument {ball} gets replaced with its assigned value

**2. f-strings**

- A f-string is a string literal that is prefixed with "f".
- These strings may contain replacement fields, which are expressions enclosed within curly braces {}.
- The expressions are replaced with their values.
- An f at the beginning of the string tells Python to allow any currently valid variable names within the string.

**Program 2.3: Code to Demonstrate the Use of f-strings with print() Function**

```
1. country = input("Which country do you live in?")
2. print(f"I live in {country}")
```

OUTPUT

```
Which country do you live in? India
I live in India
```

**Program 2.4: Given the Radius, Write Python Program to Find the Area and Circumference of a Circle**

```
1. radius = int(input("Enter the radius of a circle"))
2. area_of_a_circle = 3.1415 * radius * radius
3. circumference_of_a_circle = 2 * 3.1415 * radius
4. print(f"Area={area_of_a_circle}and Circumference={circumference_of_a_circle}")
```

OUTPUT

```
Enter the radius of a circle 5
Area = 78.53750000000001 and Circumference = 31.415000000000003
```

## Type Conversions

Explicitly casting, or converting, a variable from one type to another is called type conversion.

### 1. The int() Function

To explicitly convert a float number or a string to an integer, cast the number using int() function.

Example:

```
num=int(3.5)
n=int("1")
print(f " Float to int value is {num}")
print(f " String to int value is {n}")
```

Output

```
Float to int value is 3
String to int value is 1
```

### 2. The float() Function

The float() function returns a floating point number constructed from a number or string.

Example: Program to Demonstrate float() Casting Function

```
f1 = float(4)
f2 = float("1") #number treated as string
print(f"After Integer to Float Casting the result is {f1}")
print(f"After String to Float Casting the result is {f2}")
```

SBC KARKALA

Output
After Integer to Float Casting the result is 4.0
After String to Float Casting the result is 1.0

### 3. The str() Function

The str() function returns a string which is fairly human readable.

Example: Program to Demonstrate str() Casting Function

```
str1 = str(8)
str2 = str(3.5)
print(f"After Integer to String Casting the result is {str1}")
print(f"After Float to String Casting the result is {str2}")
```

Output
After Integer to String Casting the result is 8
After Float to String Casting the result is 3.5

### 4. The chr() Function

Convert an integer to a string of one character whose ASCII code is same as the integer using chr() function. The integer value should be in the range of 0–255.

Example: Program to Demonstrate chr() Casting Function

```
ascii_to_char = chr(100)
print(f'Equivalent Character for ASCII value of 100 is {ascii_to_char}')
```

Output
Equivalent Character for ASCII value of 100 is d

An integer value corresponding to an ASCII code is converted to the character and printed.

### 5. The complex() Function

Use complex() function to print a complex number with the value real + imag*j or convert a string or number to a complex number. If the first argument for the function is a string, it will be interpreted as a complex number and the function must be called without a second parameter. The second parameter can never be a string. Each argument may be any numeric type (including complex). If imag is omitted, it defaults to zero and the function serves as a numeric conversion function like int(), long() and float(). If both arguments are omitted, the complex() function returns 0j.

Example: Program to Demonstrate complex() Casting Function

```
c1 = complex("1")
c2 = complex(5, 8)
print(f"Result after using string in real part {c1}")
print(f"Result after using numbers in real and imaginary part {c2}")
```

Output
Result after using string in real part (1+0j)
Result after using numbers in real and imaginary part (5+8j)

SBC KARKALA

### 6. The ord() Function

The ord() function returns an integer representing Unicode code point for the given Unicode character.

Example: Program to Demonstrate ord() Casting Function

```
u4 = ord('4')
uz = ord("Z")
u_hash = ord("#")
print(f"Unicode code point for integer value of 4 is {u4}")
print(f"Unicode code point for alphabet 'Z' is {uz}")
print(f"Unicode code point for character '$' is {u_hash}")
```

Output

```
Unicode code point for integer value of 4 is 52
Unicode code point for alphabet 'Z' is 90
Unicode code point for character '$' is 35
```

### 7. The hex() Function

Convert an integer number (of any size) to a lowercase hexadecimal string prefixed with "0x" using hex() function.

Example: Program to Demonstrate hex() Casting Function

```
int_to_hex = hex(255)
print(f"After Integer to Hex Casting the result is {int_to_hex}")
```

Output

```
After Integer to Hex Casting the result is 0xff
```

### 8. The oct() Function

Convert an integer number (of any size) to an octal string prefixed with "0o" using oct() function.

Example: Program to Demonstrate oct() Casting Function

```
int_to_oct = oct(255)
print(f"After Integer to Hex Casting the result is {int_to_oct}")
```

Output

```
After Integer to Hex Casting the result is 0o377
```

### The type() Function and Is Operator

The syntax for type() function is, type(object)

The type() function returns the data type of the given object.

```
>>> type(1)
<class 'int'>

>>> type(6.4)
<class 'float'>

>>> type("A")
<class 'str'>

>>> type(True)
<class 'bool'>
```

SBCKARKALA

The type() function comes in handy if you forget the type of variable or an object during the course of writing programs.

**Identity operators**

The operators **is** and **is not** are identity operators. Operator **is** evaluates to True if the values of operands on either side of the operator point to the same object and False otherwise. The operator is not evaluates to False if the values of operands on either side of the operator point to the same object and True otherwise.

```
>>> x = "Seattle"
>>> y = "Seattle"
>>> x is y
True
```

**Dynamic and Strongly Typed Language**

Python is a dynamic language as the type of the variable is determined during run-time by the interpreter. Python is also a strongly typed language as the interpreter keeps track of all the variables types. In a strongly typed language, you are simply not allowed to do anything that's incompatible with the type of data you are working with. For example,

```
>>> 5 + 10
15
```

Here values which are of integer types are added and they are compatible.

```
>>> 1 + "a"
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Here when we try to add 1, which is an integer type with "a" which is string type, then it results in Traceback as they are not compatible.

In Python, **Traceback is printed when an error occurs**. The last line tells us the kind of error that occurred which in our case is the unsupported operand type(s).

**********

SBC KARKALA

# Chapter 3- Control Flow Statements

The term control flow details the direction the program takes. The control flow statements in Python Programming Language are

**1. Sequential Control Flow Statements**: This refers to the line by line execution, in which the statements are executed sequentially, in the same order in which they appear in the program.

**2. Decision Control Flow Statements:** Depending on whether a condition is True or False, the decision structure may skip the execution of an entire block of statements or even execute one block of statements instead of other (if, if…else and if…elif…else).

**3. Loop Control Flow Statements**: This is a control structure that allows the execution of a block of statements multiple times until a loop termination condition is met (for loop and while loop). Loop Control Flow Statements are also called Repetition statements or Iteration statements.

## 1. The if Decision Control Flow Statement

**Syntax :**

```
if condition:
    # body of if statement
```

The if decision control flow statement starts with if keyword and ends with a colon. The expression in an if statement should be a Boolean expression.
The if statement evaluates condition.
If condition is evaluated to True, the code inside the body of if is executed.
If condition is evaluated to False, the code inside the body of if is skipped.

**Example : Program check if number is greater than 0.**

```
number = 10
if number > 0:
    print('Number is positive.')
print('The if statement is easy')
```

**Output**

```
Number is positive.
The if statement is easy
```

## 2. The if…else Decision Control Flow Statement

An if statement can also be followed by an else statement which is optional. The if…else statement allows for a two-way decision.

**Syntax:**

```
if condition:
    # block of code if condition is True
else:
    # block of code if condition is False
```

The if...else statement evaluates the given condition:
If the condition evaluates to True, the code inside if is executed.
If the condition evaluates to False, the code inside else is executed.

**Example: Program to check whether the given number is positive or negative.**

| | |
|---|---|
| number = 10<br>if number > 0:<br>  print('Positive number')<br>else:<br>  print('Negative number')<br>print('This statement is always executed') | **Output**<br>Positive number<br>This statement is always executed |

### 3. The if…elif…else Decision Control Statement

The if…elif…else is also called as multi-way decision control statement. It is used when we need to choose from several possible alternatives.

**Syntax:**

```
if condition1:
    # code block 1
elif condition2:
    # code block 2
else:
    # code block 3
```

Here,
If condition1 evaluates to true, code block 1 is executed.
If condition1 evaluates to false, then condition2 is evaluated.
If condition2 is true, code block 2 is executed.
If condition2 is false, code block 3 is executed.

**Example: Program to check whether the given number is Zero, Positive or Negative.**

| | |
|---|---|
| number = 0<br>if number > 0:<br>  print("Positive number")<br>elif number == 0:<br>  print('Zero')<br>else:<br>  print('Negative number')<br>print('This statement is always executed') | **Output**<br>Zero<br>This statement is always executed |

### 4. Nested if Statement

We can also use an if statement inside of an if statement. This is known as a nested if statement.

**Syntax:**

```
# outer if statement
if condition1:
    # statement(s)
    # inner if statement
    if condition2:
        # statement(s)
    else:
        #Statements
else:
    #Statements
```

SBCKarkala

If condition1 is evaluated to True, then inner if condition2 will be evaluated. If condition2 is false then inner else block statements will be executed.
If condition1 is false the outer else block will be executed.

**Example:**

```
number = 5
# outer if statement
if (number >= 0):
    # inner if statement
    if number == 0:
        print('Number is 0')
    # inner else statement
    else:
        print('Number is positive')

# outer else statement
else:
    print('Number is negative')
```

**Output:**

```
Number is positive
```

## 5. The while Loop

In programming, loops are used to repeat a block of code. Python while loop is used to run a block code until a certain condition is met.

**Syntax:**

```
while condition:
    # body of while loop
```

- A while loop evaluates the condition
- If the condition evaluates to True, the code inside the while loop is executed.
- condition is evaluated again.
- This process continues until the condition is False.
- When condition evaluates to False, the loop stops.

**Example: Python Program to Display First 5 Numbers**

```
i = 1
while i <= 5:
    print(f"Current value of i is {i}")
    i = i + 1
```

**Output**

```
Current value of i is 1
Current value of i is 2
Current value of i is 3
Current value of i is 4
Current value of i is 5
```

## 6. The for Loop

In Python, the for loop is used to run a block of code for a certain number of times. It is used to iterate over any sequences such as list, tuple, string, etc.

**Syntax:**

```
for val in sequence:
    # statement(s)
```

Here, val accesses each item of sequence on each iteration. Loop continues until we reach the last item in the sequence.

SBCKarkala

**Example: Program to print numbers from 0 to 3 using range().**

| | |
|---|---|
| values = range(4)<br>for i in values:<br>  print(i) | **Output**<br>0<br>1<br>2<br>3 |

**range() function**

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

**Syntax**

**range(start, stop, step)**

| Parameter | Description |
|---|---|
| start | Optional. An integer number specifying at which position to start. Default is 0 |
| stop | Required. An integer number specifying at which position to stop (not included). |
| step | Optional. An integer number specifying the incrementation. Default is 1 |

**Example : Create a sequence of numbers from 3 to 5, and print each item in the sequence:**

| | |
|---|---|
| x = range(3, 6)<br>for n in x:<br>print(n) | **Output**<br>3<br>4<br>5 |

**Example: Create a sequence of numbers from 3 to 9, but increment by 2**

| | |
|---|---|
| x = range(3, 10, 2)<br>for n in x:<br>  print(n) | **Output**<br>3<br>5<br>7<br>9 |

**The continue and break Statements**

The **break** is a keyword in python which is used to bring the program control out of the loop. The break is commonly used in the cases where we need to break the loop for a given condition.

**Syntax:**

```
#loop statements
      break
```

**Example:**

| | |
|---|---|
| str = "python"<br>for i in str:<br>  if i == 'o':<br>  break<br>  print(i); | **Output**<br>p<br>y<br>t<br>h |

SBCK Karkala

The **continue** keyword return control of the iteration to the beginning of the Python for loop or Python while loop. All remaining lines in the prevailing iteration of the loop are skipped by the continue keyword, which returns execution to the beginning of the next iteration of the loop.

**Syntax:**

```
#loop statements
    continue
```

**Example:**

| | |
|---|---|
| for var in "Good":<br>  if var == "o":<br>  continue<br>  print(var) | **Output**<br>G<br>d |

**exit() Function**

We can use the in-built exit() function to quit and come out of the execution loop of the program in Python.

Syntax: exit()

exit() is defined in site module and it works only if the site module is imported so it should be used in the interpreter only.

Example: Program to exit when the value is 5.

| | |
|---|---|
| for i in range(10):<br>  if i == 5:<br>  print(exit)<br>  exit()<br>  print(i) | **Output:**<br>0<br>1<br>2<br>3<br>4<br>Use exit() or Ctrl-D (i.e. EOF) to exit |

SBC KARKALA

# Chapter 4 - Catching Exceptions Using try and except Statement

There are at least two distinguishable kinds of errors:

1. Syntax Errors
2. Exceptions

**Syntax Errors**

Syntax errors, also known as parsing errors, are perhaps the most common kind of complaint you get while you are still learning Python. For example,

| | |
|---|---|
| while True<br>print("Hello World”) | Output<br>File "<ipython-input-3-c231969faf4f>", line 1<br>while True<br>^<br>SyntaxError: invalid syntax |

The error is caused by a missing colon (':').

**Exceptions**

An exception is an unwanted event that interrupts the normal flow of the program. Errors detected during execution are called exceptions.

When an exception occurs in the program, execution gets terminated. In such cases, we get a system-generated error message. However, these exceptions can be handled in Python. By handling the exceptions, we can provide a meaningful message to the user about the issue rather than a system-generated message, which may not be understandable to the user.

Exceptions can be either built-in exceptions or user-defined exceptions. The interpreter or built-in functions can generate the built-in exceptions while user-defined exceptions are custom exceptions created by the user.

When the exceptions are not handled by programs it results in error messages as shown below.

| **Example :1** | **Example :2** |
|---|---|
| >>> 10 * (1/0) | >>> '2' + 2 |
| Traceback (most recent call last): | Traceback (most recent call last): |
| File "<stdin>", line 1, in <module> | File "<stdin>", line 1, in <module> |
| ZeroDivisionError: division by zero | TypeError: Can't convert 'int' object to str implicitly |

**Exception Handling Using try…except…finally**

Handling of exception ensures that the flow of the program does not get interrupted when an exception occurs which is done by trapping run-time errors. Handling of exceptions results in the execution of all the statements in the program.

It is possible to write programs to handle exceptions by using try…except…finally statements.

**Syntax:**

```
try:
        statement1
except ExceptionName1:
        statement2
except ExceptionName2:
        statement3
.
.
.
else:
        statement4
finally:
        statement5
```

**try .. except block**

- The try clause contains the code that can raise an exception, while the except clause contains the code lines that handle the exception.
- There can be one or more except blocks. Multiple except blocks with different exception names can be chained together. The except blocks are evaluated from top to bottom in your code, but only one except block is executed for each exception that is thrown. The first except block that specifies the exact exception name of the thrown exception is executed.
- Instead of having multiple except blocks with multiple exception names for different exceptions, you can combine multiple exception names together separated by a comma (also called parenthesized tuples) in a single except block. The syntax for combining multiple exception names in an except block is,

```
except (Exception_Name_1, Exception_Name_2, Exception_Name_3):
    statement(s)
```

**else block**

- The try…except statement has an optional else block.
- The else block lets you execute code when there is no error.

**Finally block**

- The try statement has another optional block which is intended to define clean-up actions that must be executed under all circumstances.
- A finally block is always executed before leaving the try statement, whether an exception has occurred or not.
- When an exception has occurred in the try block and has not been handled by an except block, it is re-raised after the finally block has been executed.
- The finally clause is also executed "on the way out" when any other clause of the try statement is left via a break, continue or return statement.

**Example: Program to Check for ZeroDivisionError Exception**

```
x=int(input("Enter first number :"))
y=int(input("Enter second number :"))
try:
        result = x / y
except ZeroDivisionError:
        print("Division by zero!")
else:
        print(f"Result is {result}")
finally:
                print("Executing finally clause")
```

**Output 1**

```
Enter first number :9
Enter second number :0
Division by zero!
Executing finally clause
```

**Output 2**

```
Enter first number :8
Enter second number :4
Result is 2.0
Executing finally clause
```

**Output 3**

```
Enter first number :p
Traceback (most recent call last):
  File "d:/Python Programs/IV semester/Unit 1/2test.py", line 1, in <module>
    x=int(input("Enter first number :"))
ValueError: invalid literal for int() with base 10: 'p'
```

SBCKARKALA

# Chapter 5 – Functions

A **function** is a block of code which only runs when it is called. We can pass data, known as parameters, into a function. A function can return data as a result.

**Types of function**

There are two types of function in Python programming:

**Built-in functions** - These are built-in functions in Python that are available to use.

**User-defined functions** - We can create our own functions based on our requirements.

**Few Built-in Functions**

1. **abs() :** Returns the absolute value of a number.
   **Example:**

```
x = abs(-7.25)
print(x)
Output:
7.25
```

2. **min() :** Returns the smallest item in an iterable.
   **Example:**

```
x = min(5, 10)
print(x)
Output:
5
```

3. **max() :**Returns the largest item in an iterable.
   **Example:**

```
x = max(5, 10)
print(x)
Output:
10
```

4. **divmod() :** Returns the quotient and the remainder when argument1 is divided by argument2
   **Example:**

```
x = divmod(5, 2)
print(x)
Output:
(2, 1)
```

5. **len() :** Returns the length of an object.
   **Example:**

```
mylist = ["apple", "orange", "cherry"]
x = len(mylist)
print(x)
Output:
3
```

SBCKARKALA

6. **pow() :** Returns the value of x to the power of y
   **Example:**

```
x = pow(2, 3)
print(x)
Output:
8
```

## Commonly Used Modules

**Modules** in Python are reusable libraries of code having .py extension, which implements a group of methods and statements. Python comes with many built-in modules as part of the standard library.

To use a module in your program, import the module using import statement. All the import statements are placed at the beginning of the program.

The syntax for import statement is, **import module_name**

The syntax for using a function defined in a module is, **module_name.function_name()**

The module name and function name are separated by a dot.

| import math | Output |
|---|---|
| print(math.ceil(5.4)) | 6 |
| print(math.sqrt(4)) | 2.0 |
| print(math.pi) | 3.141592653589793 |
| print(math.cos(1)) | 0.5403023058681398 |
| print(math.factorial(4)) | 24 |
| print(math.pow(2,3)) | 8.0 |

The built-in function **dir()** returns a sorted list of comma separated strings containing the names of functions, classes and variables as defined in the module. For example, you can find all the functions supported by the math module by passing the module name as an argument to the dir() function.

>>> dir(math)

['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign', 'cos', 'cosh', 'degrees', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'pi', 'pow', 'radians', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc']

Another built-in function you will find useful is the **help()** function which invokes the built-in help system. The argument to the help() function is a string, which is looked up as the name of a module, function, class, method, keyword, or documentation topic, and then a related help page is printed in the console.

**Example:**

```
import math
help(math.gcd)
```

**Output:**
Help on built-in function gcd in module math:
gcd(x, y, /)
greatest common divisor of x and y

The **random** module which generates random numbers.
>>> import random
>>> print(random.random())
0.2551941897892144
>>> print(random.randint(5,10))
9

**Third-party modules** or libraries can be installed and managed using Python's package manager pip.
The syntax for pip is, **pip install module_name**

**Arrow** is a popular Python library that offers a sensible, human-friendly approach to creating, manipulating, formatting and converting dates, times, and timestamps.

To install the arrow module, open a command prompt window and type the below command from any location.
**C:\> pip install arrow**
**Using arrow module functions:**
**>>> import arrow**
**>>> a = arrow.utcnow()**
**>>> a.now()**
**<Arrow [2017-12-23T20:45:14.490380+05:30]>**

**UTC** is the time standard commonly used across the world. Current date including time is displayed using **now()** function.

## Function Definition and Calling the Function

User-defined functions are reusable code blocks created by users to perform some specific task in the program.
The syntax for function definition is,

```
def function_name(parameter_1, parameter_2, …, parameter_n):
        statement(s)
```

def - keyword used to declare a function
function_name - any name given to the function
parameter_1, parameter_2, …, parameter_n - any value passed to function
A colon is required at the end of the function header.
Block of statements that defines the body of the function.

**Function Call**
Calling the function actually performs the specified actions with the indicated parameters.
The syntax for function call or calling function is,

**function_name(argument_1, argument_2,…,argument_n)**

Arguments are the actual value that is passed into the calling function. There must be a one to one correspondence between the formal parameters in the function definition and the actual arguments of the calling function.

When a Python interpreter reads a Python file, it first sets a few special variables. Then it executes the code from the file. One of those variables is called _ _name_ _ .

**__name__**is a special variable. __name__is a built-in variable which evaluates to the name of the current module.

- If the source file is executed as the main program, the interpreter sets the __name__ variable to have a value "__main__".
- If this file is being imported from another module, __name__will be set to the module's name.

**Example 1:**

```
#Program to Demonstrate a Function with and without Arguments
def function1():
        print("This is a function definition with NO Argument")
def function2(message):
        print(f"This is a function definition with {message}")
def main():
        function1()
        function2("One Argument")
if __name__== "__main__":
        main()
```

```
Output
This is a function definition with NO Argument
This is a function definition with One Argument
```

**Example 2:**

```
#Program to add two numbers using user-defined function
def sum(a,b):
        print(f"The sum is : {a+b}")
def main():
        sum(5,6)
if __name__== "__main__":
        main()
```

```
Output
The sum is : 11
```

**The return Statement and void Function**

The return keyword is to exit a function and return a value.

**Syntax: return [expression_list]**

The return statement terminates the execution of the function definition in which it appears and returns control to the calling function. It can also return an optional value to its calling function.

In Python, it is possible to define functions without a return statement. Functions like this are called **void functions**, and they return None.

To **return multiple values** from a function, they must be separated by a comma which by default is constructed as a tuple by Python. Extracting the returned tuple values back into variables is called "tuple unpacking".

SBCKarkala

**Example**

```
#Program to demonstrate returning multiple values from a function
def arithmetic(a,b):
    s=a+b
    p=a*b
    q=a/b
    return s,p,q
def main():
    s,p,q=arithmetic(5,3)
    print(f"The sum is : {s}")
    print(f"The Product is : {p}")
    print(f"The Quotient is : {q}")
if __name___== "__main__":
        main()
```

```
Output:
The sum is : 8
The Product is : 15
The Quotient is : 1.6666666666666667
```

## Scope and Lifetime of Variables

Python programs have two scopes: global and local.

**Global variables**

- A variable is a global variable if its value is accessible and modifiable throughout your program.
- Global variables have a global scope.

**Local variable**

- A variable that is defined inside a function definition is a local variable.
- The lifetime of a variable refers to the duration of its existence.
- The local variable is created and destroyed every time the function is executed, and it cannot be accessed by any code outside the function definition.
- Local variables inside a function definition have local scope and exist as long as the function is executing.

It is possible to access global variables from inside a function, as long as you have not defined a local variable with the same name. A local variable can have the same name as a global variable, but they are totally different so changing the value of the local variable has no effect on the global variable. Only the local variable has meaning inside the function in which it is defined.

**Nested Function**

You can nest a function definition within another function definition. A nested function (inner function definition) can "inherit" the arguments and variables of its outer function definition. In other words, the inner function contains the scope of the outer function. The inner function can use the arguments and variables of the outer function, while the outer function cannot use the arguments and variables of the inner function. The inner function definition can be invoked by calling it from within the outer function definition.

SBC KARKALA

**Example:**

**#Program to Demonstrate the Scope of Variables and nested function**

```
test_variable = 5
def outer_function():
    test_variable = 60
    def inner_function():
        test_variable = 100
        print(f"Local variable value of {test_variable} have scope in inner function")
    inner_function()
    print(f"Local variable value of {test_variable} have local scope in outer function ")

outer_function()
print(f"Global variable value of {test_variable} is displayed ")
```

```
Output
Local variable value of 100 have scope in inner function
Local variable value of 60 have local scope in outer function
Global variable value of 5 is displayed
```

## Default Parameters

Python allows function arguments to have default values. If the function is called without the argument, the argument gets its default value. The default value is assigned by using the assignment(=) operator of the form keywordname=value.

**Example: #Program to Demonstrate the Use of Default Parameters**

```
def work_area(prompt, domain="Data Analytics"):
    print(f"{prompt} {domain}")
work_area("Sam works in")
work_area("Alice has interest in", "Internet of Things")
```

```
Output:
Sam works in Data Analytics
Alice has interest in Internet of Things
```

There are two parameters in the function for work_area(). For the first parameter prompt, we have to specify the argument in the function call. For the second parameter domain, a default value has been set. In the function call, if we skip the second argument then the default value will be set. If we specify the second argument in the function call then the default value will be overwritten with the latest value as specified in the second argument.

## Keyword Arguments

We can send arguments with the kwarg = value syntax. This way the order of the arguments does not matter. We explicitly specify the argument name along with their value in the form kwarg=value.

**Example:**

```
def my_function(child3, child2, child1):
    print("The youngest child is " + child3)

my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

```
Output:
The youngest child is Linus
```

***args and **kwargs**

*args and **kwargs allows you to pass a variable number of arguments to the calling function. They are used when the user does not know in advance about how many arguments will be passed to the calling function.

*args as parameter in function definition allows you to pass a non-keyworded, variable length tuple argument list to the calling function. *args must come after all the positional parameters.

**kwargs as parameter in function definition allows you to pass keyworded, variable length dictionary argument list to the calling function. **kwargs must come right at the end.

**Example to demonstrate *args**

```
def myFun(*argv):
  for arg in argv:
      print(arg)

myFun('Hello', 'Welcome', 'to', 'Python','Programming')
```

```
Output:
Hello
Welcome
to
Python
Programming
```

**Example to demonstrate **kwargs**

```
def myFun(**kwargs):
   for key, value in kwargs.items():
      print("%s == %s" % (key, value))

myFun(first='Learning', mid=' Python ', last=' Programming ')
```

```
Output:
first == Learning
mid ==  Python
last ==  Programming
```

## Command Line Arguments

A Python program can accept any number of arguments from the command line. Command line arguments is a methodology in which user will give inputs to the program through the console using commands. You need to import sys module to access command line arguments. All the command line arguments in Python can be printed as a list of string by executing sys.argv. It is a list of command line arguments.

len(sys.argv) provides the number of command line arguments.

sys.argv[0] is the name of the current Python script.

SBC KARKALA

```
#Program to take command line arguments and display them
import sys
n=len(sys.argv)
args=sys.argv
print(f"Number of command line aregs={n}”)
print(f"The arguments are :{args}")
print("The args one by one :")
for a in args:
    print(a)

Program is executed at the command prompt.
```

**Output:**

**D:\Python Programs\Demo programs>python Commandline.py 101 India Yellow 560**

Number of command line aregs= 5

The arguments are : ['Commandline.py', '101', 'India', 'Yellow', '560']

The args one by one :

Commandline.py

101

India

Yellow

560

**********

SBCKARKALA